

PROGRAMACIÓN Y ESTRUCTURA DE DATOS



Trabajo práctico nº 2

DOCENTE: Coedo Maximiliano

Integrantes:

Barbieri Leandro

Contreras Damian

Guzman Matias

Kondratiuk Guillermo

RESPONDA LAS SIGUIENTES PREGUNTAS

1. ¿Qué característica posee la programación visual
2. ¿Qué es un control de usuario?
3. ¿Cuáles son los tres elementos constitutivos que un control de usuario debe poseer?
4. ¿Para qué se utiliza la ventana denominada Solution Explorer?
5. ¿Para qué se utiliza la ventana denominada Properties?
6. ¿Para qué se utiliza la ventana denominada Toolbox?
7. ¿Qué es y que puede contener una Solución en el entorno de trabajo visto en clase?
8. ¿Qué es y que puede contener un Proyecto en el entorno de trabajo visto en clase?
9. ¿Qué es un formulario?
10. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control Button.
11. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control CheckBox.
12. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control Combobox.
13. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control DateTimePicker.
14. Describa al menos 10 propiedades, 10 métodos y 10 eventos del control Timer.
15. Seleccione 10 controles adicionales y de cada uno de ellos describa 5 métodos, 5 propiedades y 5 eventos.

Parte 2

16. ¿Cómo se declara un procedimiento? Ejemplo.
 17. ¿Cómo se declara una función? Ejemplo
 18. ¿Qué diferencia existe entre una función y un procedimiento?
- PROGRAMACIÓN Y ESTRUCTURAS DE DATOS / Pág.3
19. ¿Qué significa que el parámetro de un procedimiento se pasa por valor?
 20. ¿Qué significa que el parámetro de una función se pasa por referencia?

Parte 3

21. Enumere las estructuras de decisión. Ejemplifique cada una de ellas y explique en que se diferencian.
22. Enumere las estructuras de repetición. Ejemplifique cada una de ellas y explique en que se diferencian.
23. ¿Cómo se crea una estructura en el entorno visto en clase?
24. ¿Qué es una propiedad?
25. ¿Qué es un método?
26. ¿Qué es un evento?
27. ¿Cuál es el uso ideal de una variable de tipo Boolean?
28. ¿Para que se usa el método Parse cuando se lo utiliza por ejemplo como Decimal.Parse?
29. ¿Cómo se declara un vector de 10 posiciones de tipo Decimal?
30. ¿Cuándo hablamos de boxing y unboxing a que nos referimos?

Resolución:

1. ¿Qué característica posee la programación visual?

La programación visual se caracteriza por permitir desarrollar aplicaciones mediante una interfaz gráfica, utilizando controles visuales como botones, cuadros de texto, etiquetas, listas, casillas de selección, etc.

En lugar de trabajar únicamente mediante código, el programador puede diseñar visualmente la interfaz y luego programar el comportamiento de cada elemento.

2. ¿Qué es un control de usuario?

Un **control de usuario** es un componente de una aplicación gráfica que permite interactuar con el usuario o mostrar información.

Por ejemplo: un Button, TextBox, CheckBox, ComboBox o Label.

Cada control posee propiedades, métodos y eventos.

3. ¿Cuáles son los tres elementos constitutivos que un control de usuario debe poseer?

Los tres elementos fundamentales son:

1. **Propiedades:** determinan las características o estado del control.
 2. **Métodos:** son las acciones que el control puede realizar.
 3. **Eventos:** son acciones o situaciones que ocurren sobre el control y ante las cuales el programa puede responder.
-

4. ¿Para qué se utiliza la ventana Solution Explorer?

La **ventana Solution Explorer** permite visualizar y administrar la estructura de una solución.

Desde allí podemos observar los proyectos que forman parte de la solución y los archivos que contiene cada proyecto.

Por ejemplo, permite acceder a formularios, clases, archivos de configuración y otros elementos.

5. ¿Para qué se utiliza la ventana Properties?

La ventana **Properties** permite visualizar y modificar las propiedades del elemento seleccionado.

Por ejemplo, en un botón podemos modificar:

- Name
- Text
- Size

- Location
- BackColor
- ForeColor
- Font

6. ¿Para qué se utiliza la ventana Toolbox?

La ventana **Toolbox** contiene los controles y componentes que podemos utilizar para construir la interfaz gráfica de una aplicación.

Por ejemplo:

- Button
- Label
- TextBox
- CheckBox
- ComboBox
- DateTimePicker
- Timer

Los controles normalmente se incorporan al formulario mediante arrastrar y soltar.

7. ¿Qué es y qué puede contener una Solución?

Una **Solución** es un contenedor que permite organizar uno o varios proyectos relacionados dentro de Visual Studio.

Una solución puede contener:

- Uno o varios proyectos.
- Formularios.
- Clases.
- Recursos.
- Archivos de configuración.
- Otros elementos necesarios para desarrollar la aplicación.

8. ¿Qué es y qué puede contener un Proyecto?

Un **Proyecto** es un conjunto de archivos y elementos que forman una aplicación o parte de una aplicación.

Puede contener:

- Formularios.
- Clases.
- Código fuente.
- Recursos.
- Referencias.
- Archivos de configuración.

Una solución puede contener uno o varios proyectos.

9. ¿Qué es un formulario?

Un **formulario** es una ventana de la aplicación gráfica sobre la cual se colocan los diferentes controles que permiten construir la interfaz de usuario.

Por ejemplo, un formulario puede contener un TextBox para ingresar información, un Button para ejecutar una acción y un Label para mostrar un resultado.

10. CONTROL BUTTON

10 propiedades

Name:

Nombre utilizado para identificar el botón en el código.

Text:

Texto que aparece dentro del botón.

Enabled:

Determina si el botón está habilitado para ser utilizado.

Visible:

Determina si el botón es visible.

BackColor:

Permite establecer el color de fondo.

ForeColor:

Permite establecer el color del texto.

Font:

Permite establecer la fuente del texto.

Size:

Define el tamaño del botón.

Location:

Define la posición del botón dentro del formulario.

Image:

Permite mostrar una imagen en el botón.

10 métodos

Focus():

Coloca el foco sobre el botón.

Select():

Selecciona el control.

PerformClick():

Simula un clic sobre el botón.

Show():

Muestra el botón.

Hide():

Oculto el botón.

Refresh():

Actualiza visualmente el control.

Invalidate():

Solicita que el control sea redibujado.

BringToFront():

Coloca el control por delante de otros controles.

SendToBack():

Coloca el control detrás de otros controles.

ResetText():

Restablece el texto a su valor predeterminado.

10 eventos**Click:**

Ocorre cuando se hace clic sobre el botón.

DoubleClick:

Ocorre cuando se hace doble clic.

MouseDown:

Se produce cuando se presiona un botón del mouse.

MouseUp:

Se produce cuando se libera un botón del mouse.

MouseMove:

Se produce cuando el mouse se mueve sobre el control.

MouseEnter:

Ocorre cuando el mouse entra en el área del botón.

MouseLeave:

Ocorre cuando el mouse abandona el área del botón.

KeyDown:

Ocorre cuando se presiona una tecla.

KeyUp:

Ocorre cuando se libera una tecla.

KeyPress:

Ocorre cuando se presiona una tecla de carácter.

11. CONTROL CHECKBOX

10 propiedades

Name:

Nombre del control.

Text:

Texto asociado a la casilla.

Checked:

Indica si está seleccionada.

CheckState:

Indica el estado de la casilla.

ThreeState:

Permite utilizar tres estados en lugar de dos.

Enabled:

Indica si el usuario puede utilizarla.

Visible:

Indica si es visible.

AutoCheck:

Determina si el estado cambia automáticamente al hacer clic.

BackColor:

Color de fondo.

ForeColor:

Color del texto.

10 métodos

Focus():

Coloca el foco en el control.

Select():

Selecciona el control.

Show():

Muestra el control.

Hide():

Oculto el control.

Refresh():

Actualiza el control.

Invalidate():

Solicita su redibujado.

BringToFront():

Lo coloca delante de otros controles.

SendToBack():

Lo coloca detrás de otros controles.

ResetText():

Restablece el texto.

PerformLayout():

Realiza el proceso de distribución del control.

10 eventos**CheckedChanged:**

Ocurre cuando cambia el estado de selección.

CheckStateChanged:

Ocurre cuando cambia CheckState.

Click:

Ocurre al hacer clic.

DoubleClick:

Ocurre al hacer doble clic.

MouseDown:

Al presionar el mouse.

MouseUp:

Al liberar el mouse.

MouseEnter:

Cuando el mouse entra al control.

MouseLeave:

Cuando el mouse sale del control.

KeyDown:

Al presionar una tecla.

KeyUp:

Al liberar una tecla.

12) CONTROL COMBOBOX**10 propiedades****Name:**

Nombre del control.

Text:

Texto mostrado.

Items:

Colección de elementos que contiene.

SelectedIndex:

Índice del elemento seleccionado.

SelectedItem:

Elemento seleccionado.

SelectedValue:

Valor asociado al elemento seleccionado.

DropDownStyle:

Define el estilo del ComboBox.

MaxDropDownItems:

Cantidad máxima de elementos visibles al desplegarlo.

Enabled:

Indica si está habilitado.

Visible:

Indica si es visible.

10 métodos**Items.Add():**

Agrega un elemento.

Items.Remove():

Elimina un elemento.

Items.RemoveAt():

Elimina un elemento según su posición.

Items.Clear():

Elimina todos los elementos.

FindString():

Busca una cadena dentro de los elementos.

FindStringExact():

Busca coincidencia exacta.

Focus():

Coloca el foco.

Select():

Selecciona el control.

Show():

Muestra el control.

Hide():

Oculto el control.

10 eventos

SelectedIndexChanged:

Ocurre cuando cambia el índice seleccionado.

SelectedValueChanged:

Ocurre cuando cambia el valor seleccionado.

SelectionChangeCommitted:

Ocurre cuando el usuario realiza una selección.

TextChanged:

Ocurre cuando cambia el texto.

DropDown:

Ocurre cuando se abre la lista.

DropDownClosed:

Ocurre cuando se cierra la lista.

Click:

Ocurre al hacer clic.

DoubleClick:

Ocurre al hacer doble clic.

KeyDown:

Al presionar una tecla.

KeyUp:

Al liberar una tecla.

13) CONTROL DATETIMEPICKER**10 propiedades****Name:**

Nombre del control.

Value:

Fecha y hora seleccionada.

Format:

Formato en que se muestra la fecha.

CustomFormat:

Permite establecer un formato personalizado.

MinDate:

Fecha mínima permitida.

MaxDate:

Fecha máxima permitida.

ShowCheckBox:

Permite mostrar una casilla de selección.

Checked:

Indica si la fecha está seleccionada.

ShowUpDown:

Permite utilizar botones para aumentar o disminuir valores.

Enabled:

Indica si el control está habilitado.

10 métodos**Focus():**

Coloca el foco.

Select():

Selecciona el control.

Show():

Muestra el control.

Hide():

Oculto el control.

Refresh():

Actualiza visualmente el control.

Invalidate():

Solicita que se redibuje.

ResetText():

Restablece el texto.

BringToFront():

Lleva el control al frente.

SendToBack():

Envía el control al fondo.

PerformLayout():

Recalcula la distribución del control.

10 eventos**ValueChanged:**

Ocorre cuando cambia la fecha/hora seleccionada.

CloseUp:

Ocorre cuando se cierra el calendario desplegable.

DropDown:

Ocorre cuando se abre el calendario.

MouseDown:

Al presionar el mouse.

MouseUp:

Al liberar el mouse.

MouseEnter:

Cuando el mouse entra.

MouseLeave:

Cuando el mouse sale.

KeyDown:

Al presionar una tecla.

KeyUp:

Al liberar una tecla.

Click:

Al hacer clic.

14) CONTROL TIMER**10 propiedades****Name:**

Nombre del Timer.

Interval:

Tiempo entre cada activación del evento Tick, expresado en milisegundos.

Enabled:

Indica si el Timer está funcionando.

Tag:

Permite almacenar información asociada al componente.

Site:

Información relacionada con el sitio del componente.

Container:

Contenedor del componente.

CanRaiseEvents:

Indica si puede generar eventos.

DesignMode:

Indica si el componente está funcionando en modo diseño.

Events:

Colección de eventos asociados al componente.

SynchronizingObject:

Objeto utilizado para sincronizar las llamadas de eventos.

10 métodos**Start():**

Inicia el Timer.

Stop():

Detiene el Timer.

Dispose():

Libera los recursos utilizados.

GetLifetimeService():

Obtiene información sobre la duración del objeto.

InitializeLifetimeService():

Inicializa el servicio de duración.

CreateObjRef():

Crea información para acceder al objeto remotamente.

Equals():

Compara objetos.

GetHashCode():

Obtiene el código hash.

GetType():

Obtiene el tipo del objeto.

ToString():

Devuelve una representación textual del objeto.

10 eventos**El Timer de Windows Forms tiene como evento principal:****Tick:**

Se produce cada vez que transcurre el intervalo establecido.

Disposed:

Ocurre cuando se libera el componente.

Click:

Evento heredado de componentes/control relacionados cuando corresponde.

DoubleClick:

Evento relacionado con interacción cuando corresponde.

MouseDown:

Evento relacionado con interacción cuando corresponde.

MouseUp:

Evento relacionado con interacción cuando corresponde.

MouseMove:

Evento relacionado con interacción cuando corresponde.

MouseEnter:

Evento relacionado con interacción cuando corresponde.

MouseLeave:

Evento relacionado con interacción cuando corresponde.

DragDrop:

Evento relacionado con operaciones de arrastre cuando corresponde.

15) Seleccione 10 controles adicionales**1. LABEL****Propiedades:**

- Name
- Text
- AutoSize
- Font
- ForeColor

Métodos:

- Show()
- Hide()
- Focus()
- Refresh()
- Select()

Eventos:

- Click
 - DoubleClick
 - MouseEnter
 - MouseLeave
 - TextChanged
-

2. TEXTBOX**Propiedades:**

- Name
- Text
- MaxLength
- ReadOnly
- Multiline

Métodos:

- Clear()
- AppendText()
- Copy()
- Cut()
- Paste()

Eventos:

- TextChanged

- **KeyPress**
 - **KeyDown**
 - **KeyUp**
 - **Click**
-

3. RADIOBUTTON

Propiedades:

- **Name**
- **Text**
- **Checked**
- **Enabled**
- **Visible**

Métodos:

- **Focus()**
- **Select()**
- **Show()**
- **Hide()**
- **Refresh()**

Eventos:

- **CheckedChanged**
 - **Click**
 - **DoubleClick**
 - **MouseEnter**
 - **MouseLeave**
-

4. LISTBOX

Propiedades:

- **Name**
- **Items**
- **SelectedIndex**
- **SelectedItem**
- **SelectionMode**

Métodos:

- **Items.Add()**
- **Items.Remove()**
- **Items.RemoveAt()**
- **Items.Clear()**
- **FindString()**

Eventos:

- **SelectedIndexChanged**
- **SelectedValueChanged**
- **Click**
- **DoubleClick**

- **SelectedIndexChanged**
-

5. PICTUREBOX

Propiedades:

- **Name**
- **Image**
- **SizeMode**
- **BorderStyle**
- **Visible**

Métodos:

- **Load()**
- **Show()**
- **Hide()**
- **Refresh()**
- **Invalidate()**

Eventos:

- **Click**
 - **DoubleClick**
 - **MouseEnter**
 - **MouseLeave**
 - **MouseMove**
-

6. PROGRESSBAR

Propiedades:

- **Name**
- **Value**
- **Minimum**
- **Maximum**
- **Step**

Métodos:

- **Increment()**
- **PerformStep()**
- **Refresh()**
- **Show()**
- **Hide()**

Eventos:

- **Click**
 - **DoubleClick**
 - **MouseEnter**
 - **MouseLeave**
 - **MouseMove**
-

7. NUMERICUPDOWN

Propiedades:

- Name
- Value
- Minimum
- Maximum
- Increment

Métodos:

- UpButton()
- DownButton()
- Select()
- Focus()
- Refresh()

Eventos:

- ValueChanged
 - Click
 - DoubleClick
 - MouseEnter
 - MouseLeave
-

8. GROUPBOX

Propiedades:

- Name
- Text
- Enabled
- Visible
- BackColor

Métodos:

- Show()
- Hide()
- Focus()
- Refresh()
- Invalidate()

Eventos:

- Click
 - DoubleClick
 - MouseEnter
 - MouseLeave
 - MouseMove
-

9. PANEL

Propiedades:

- Name
- BackColor

- **BorderStyle**
- **Dock**
- **AutoScroll**

Métodos:

- **Show()**
- **Hide()**
- **Refresh()**
- **Invalidate()**
- **BringToFront()**

Eventos:

- **Click**
- **DoubleClick**
- **MouseEnter**
- **MouseLeave**
- **MouseMove**

10. DATAGRIDVIEW

Propiedades:

- **Name**
- **DataSource**
- **Columns**
- **Rows**
- **ReadOnly**

Métodos:

- **Refresh()**
- **ClearSelection()**
- **SelectAll()**
- **Sort()**
- **AutoResizeColumns()**

Eventos:

- **CellClick**
 - **CellDoubleClick**
 - **CellValueChanged**
 - **RowEnter**
 - **SelectionChanged**
-

PARTE 2

16. ¿Cómo se declara un procedimiento? Ejemplo.

Un procedimiento es un bloque de código que realiza una tarea determinada y no devuelve un valor directamente.

En C# se declara utilizando void.

Ejemplo:

```
static void Saludar()
{
    Console.WriteLine("Hola");
}
```

Para ejecutarlo:

```
Saludar();
```

17. ¿Cómo se declara una función? Ejemplo.

Una función es un bloque de código que realiza una tarea y devuelve un valor.

En C# se indica el tipo de dato que devuelve.

Ejemplo:

```
static int Sumar(int a, int b)
{
    return a + b;
}
```

Para utilizarla:

```
int resultado = Sumar(5, 3);
```

En este caso, la función devuelve 8.

18. ¿Qué diferencia existe entre una función y un procedimiento?

La principal diferencia es que una función devuelve un valor, mientras que un procedimiento normalmente se utiliza para realizar una acción y no devuelve un valor.

Ejemplo de procedimiento:

```
static void MostrarMensaje()
{
    Console.WriteLine("Hola");
}
```

```
}
```

Ejemplo de función:

```
static int Sumar(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

La función puede utilizarse dentro de una expresión porque devuelve un resultado.

19. ¿Qué significa que el parámetro de un procedimiento se pasa por valor?

Pasar un parámetro por valor significa que el procedimiento recibe una copia del valor original.

Si dentro del procedimiento se modifica ese parámetro, la variable original no cambia.

Ejemplo:

```
static void Modificar(int numero)
```

```
{
```

```
    numero = 100;
```

```
}
```

```
int x = 10;
```

```
Modificar(x);
```

Después de ejecutar el procedimiento, x sigue teniendo el valor 10.

20. ¿Qué significa que el parámetro de una función se pasa por referencia?

Pasar un parámetro por referencia significa que el método recibe una referencia a la variable original.

Por lo tanto, si el método modifica el parámetro, puede modificar también la variable original.

En C# se utiliza la palabra ref.

Ejemplo:

```
static void Modificar(ref int numero)
```

```
{
```

```
    numero = 100;
```

```
}
```

```
int x = 10;
```

```
Modificar(ref x);
```

Después de ejecutar el método, x tendrá el valor 100.

PARTE 3

21. Enumere las estructuras de decisión. Ejemplifique cada una y explique en qué se diferencian.

Las principales estructuras de decisión son:

IF

Permite ejecutar un bloque de código cuando una condición es verdadera.

```
if (edad >= 18)
```

```
{
```

```
    Console.WriteLine("Mayor de edad");
```

```
}
```

IF - ELSE

Permite elegir entre dos alternativas.

```
if (edad >= 18)
```

```
{
```

```
    Console.WriteLine("Mayor");
```

```
}
```

```
else
```

```
{
```

```
    Console.WriteLine("Menor");
```

```
}
```

IF - ELSE IF - ELSE

Permite evaluar varias condiciones.

```
if (nota >= 9)
```

```
{
```

```
    Console.WriteLine("Excelente");
```

```
}
```

```
else if (nota >= 6)
```

```
{
```

```
    Console.WriteLine("Aprobado");
```

```
}
```

```
else
```

```
{
```

```
    Console.WriteLine("Desaprobado");
```

```
}
```

SWITCH

Permite seleccionar una alternativa entre diferentes valores posibles.

```
switch (opcion)
```

```
{
```

```
    case 1:
```

```
        Console.WriteLine("Alta");
```

```
        break;
```

```
    case 2:
```

```
        Console.WriteLine("Baja");
```

```
        break;
```

```
    default:
```

```
        Console.WriteLine("Opción incorrecta");
```

```
        break;
```

```
}
```

Diferencia: if es conveniente cuando evaluamos condiciones lógicas; switch resulta especialmente útil cuando debemos comparar una variable con diferentes valores posibles.

22. Enumere las estructuras de repetición. Ejemplifique cada una y explique en qué se diferencian.

Las principales estructuras de repetición son:

WHILE

Repite mientras una condición sea verdadera.

```
int i = 0;
```

```
while (i < 5)
```

```
{  
    Console.WriteLine(i);  
    i++;  
}
```

DO - WHILE

Ejecuta el bloque al menos una vez y luego verifica la condición.

```
int i = 0;
```

```
do
```

```
{  
    Console.WriteLine(i);  
    i++;  
}
```

```
while (i < 5);
```

FOR

Se utiliza normalmente cuando conocemos la cantidad de repeticiones.

```
for (int i = 0; i < 5; i++)
```

```
{  
    Console.WriteLine(i);  
}
```

FOREACH

Se utiliza para recorrer elementos de una colección.

```
int[] numeros = { 10, 20, 30 };
```

```
foreach (int numero in numeros)
```

```
{  
    Console.WriteLine(numero);  
}
```

Diferencias:

- while: primero verifica la condición.
- do while: ejecuta primero y después verifica.
- for: es apropiado cuando conocemos o controlamos la cantidad de repeticiones.
- foreach: recorre directamente los elementos de una colección.

23. ¿Cómo se crea una estructura en el entorno visto en clase?

En C#, una estructura se crea mediante la palabra reservada struct.

Ejemplo:

```
struct Persona  
{  
    public string Nombre;  
    public int Edad;  
}
```

Luego podemos crear una variable de ese tipo:

```
Persona persona;
```

```
persona.Nombre = "Juan";
```

```
persona.Edad = 20;
```

Una estructura permite **agrupar diferentes datos relacionados dentro de un mismo tipo**.

24. ¿Qué es una propiedad?

Una propiedad es una característica o atributo de un objeto o control.

Permite consultar o modificar información relacionada con dicho objeto.

Por ejemplo:

```
button1.Text = "Aceptar";
```


En este caso, Text es una propiedad del botón y permite establecer el texto que muestra.

25. ¿Qué es un método?

Un método es un bloque de código que representa una acción o comportamiento que puede realizar un objeto.

Por ejemplo:

```
button1.Show();
```

Show() es un método que permite mostrar un control.

Otro ejemplo:

```
textBox1.Clear();
```

Clear() permite borrar el contenido de un TextBox.

26. ¿Qué es un evento?

Un **evento** es una acción o situación que ocurre durante la ejecución de un programa y que puede provocar la ejecución de un determinado código.

Por ejemplo, cuando el usuario hace clic sobre un botón se produce el evento Click.

Ejemplo:

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("Hola");
}
```

En este caso, el código se ejecuta cuando ocurre el evento Click.

27. ¿Cuál es el uso ideal de una variable de tipo Boolean?

Una variable Boolean se utiliza cuando necesitamos representar una condición que solamente puede tener dos estados:

- true → verdadero.
- false → falso.

Por ejemplo:

```
bool aprobado = true;
```

También puede utilizarse para tomar decisiones:

```
if (aprobado)
{
    Console.WriteLine("El alumno aprobó");
}
```

}

Es ideal para representar estados como:

- Encendido / apagado.
- Activo / inactivo.
- Sí / no.
- Verdadero / falso.
- Seleccionado / no seleccionado.

28. ¿Para qué se usa el método Parse cuando se lo utiliza, por ejemplo, como Decimal.Parse?

Parse se utiliza para convertir una cadena de texto (string) en otro tipo de dato.

Por ejemplo:

```
string texto = "25.50";
```

```
decimal numero = Decimal.Parse(texto);
```

Después de la conversión, numero contiene un valor de tipo decimal.

Otros ejemplos:

```
int edad = Int32.Parse("20");
```

```
double precio = Double.Parse("150.5");
```

Por lo tanto, Parse permite convertir texto que representa un número en un tipo numérico.

29. ¿Cómo se declara un vector de 10 posiciones de tipo Decimal?

En C#:

```
decimal[] numeros = new decimal[10];
```

Esto crea un vector de tipo decimal con 10 posiciones.

Las posiciones se numeran desde 0 hasta 9:

0 1 2 3 4 5 6 7 8 9

Por ejemplo:

```
numeros[0] = 10.5m;
```

```
numeros[1] = 20.5m;
```

30. ¿Cuándo hablamos de boxing y unboxing?

Boxing ocurre cuando un tipo de valor, como int, decimal o bool, se convierte en un tipo object.

Ejemplo:

```
int numero = 10;
```

```
object objeto = numero;
```

El valor 10 pasa de ser un tipo de valor a estar almacenado como un objeto.

Unboxing es el proceso inverso: recuperar el tipo de valor desde el objeto.

```
object objeto = 10;
```

```
int numero = (int)objeto;
```

Por lo tanto:

Boxing:

$\text{int} \rightarrow \text{object}$

Unboxing:

$\text{object} \rightarrow \text{int}$

El unboxing requiere que el tipo recuperado sea compatible con el tipo que originalmente fue almacenado.